CO2039 – Advanced Programming
Lab 1: Student Classification System

Overview

This document specifies **Lab 1** of the course Advanced Programming (CO2039). The objective of this lab is to practice fundamental object-oriented programming (OOP) principles through a structured class design and UML-based specification.

This lab consists of **four exercises**, with a **total score of 100 points**. Each exercise focuses on a different aspect of object-oriented design and builds upon the previous ones.

In particular, this lab focuses on the following concepts:

- **Composition (HAS–A)** and **Encapsulation**, where each `Student` object owns a `Transcript` object for managing academic records and computing GPA.
- **Inheritance** and **Method Overriding**, where `RegularStudent` and `HonorsStudent` extend the abstract class `Student` and provide specialized behavior.

The class diagrams provided in this document follow standard UML notation:

- Class with name written in *italic* and marked with `«abstract»` is abstract.
- Method names written in *italic* indicate abstract methods that must be implemented by subclasses.
- Visibility modifiers are indicated using UML symbols:
  - `+` public
  - `#` protected
  - `-` private

You are expected to follow the UML specification *exactly* when implementing the classes. Additional fields or methods are not allowed unless explicitly stated. Design decisions such as data validation, method responsibilities, and access control must be consistent with the intent expressed in the diagrams.

The purpose of this lab is not only to produce working code, but also to demonstrate clean object-oriented design and disciplined use of abstraction.

Contents

1 Exercise 1: Transcript Management (30 points) 3

2 Exercise 2: Student Classification (30 points) 6

3 Exercise 3: Student Policies (20 points) 10

4 Exercise 4: Reporting and Sorting (20 points) 13

1 Exercise 1: Transcript Management (30 points)

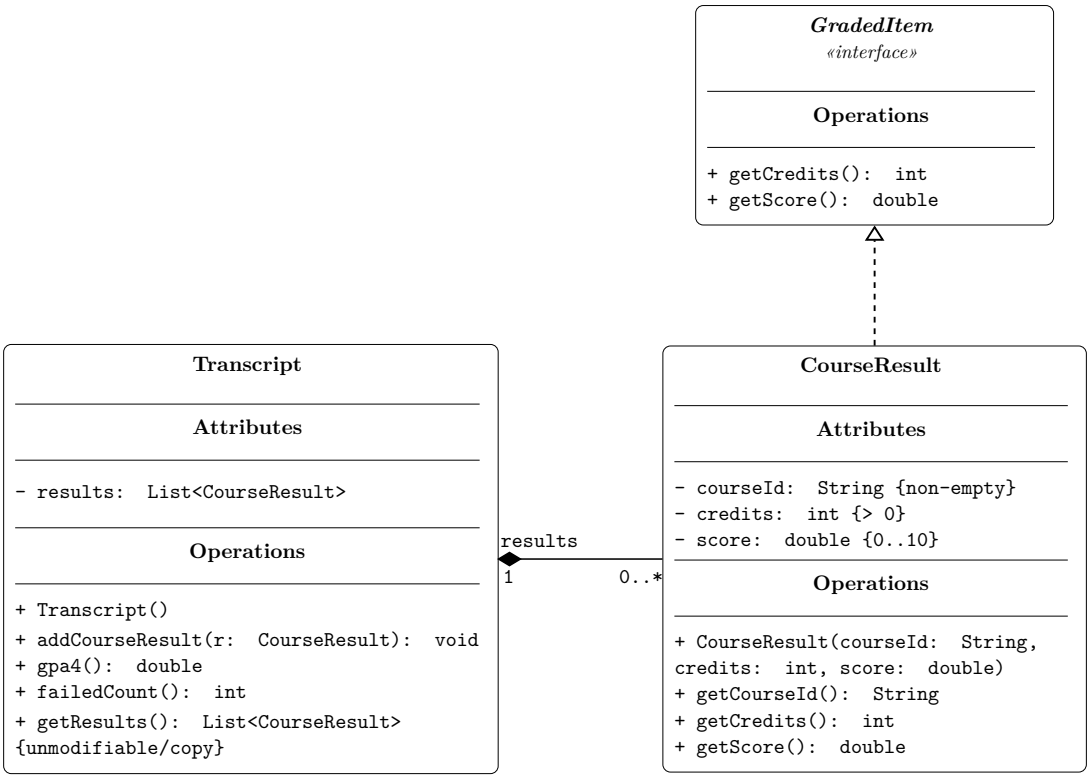

Figure 1: UML class diagram of the Transcript management system

Overview

In this exercise, you will design and implement a small academic transcript management system following the given UML class diagram in Figure 1. The goal is to practice core object-oriented programming principles, including:

- Composition (HAS-A relationships)
- Encapsulation and data protection
- Interfaces and implementation
- Separation of responsibilities between classes

The system models how a transcript stores course results and computes academic statistics such as GPA and the number of failed courses. You must follow the structure and constraints shown in the class diagram. Any deviation from the specified design may result in point deductions.

Tasks

1. **Define the interface `GradedItem`.**

   Create an interface `GradedItem` representing an assessable academic item. The interface must declare the following methods:

- `int getCredits();`
- `double getScore();`

The interface must not contain any fields or implementation logic. It defines the minimum contract required for GPA computation.

2. **Implement the class `CourseResult`.**

   Implement a class `CourseResult` that represents the result of a single course and *implements* the `GradedItem` interface.

   The class must contain the following attributes:

   - `courseId : String` (must be non-empty),
   - `credits : int` (must be greater than 0),
   - `score : double` (must be in the range $[0.0, 10.0]$).

   All attributes must be private and validated in the constructor. Provide getter methods for all attributes. No setter methods are allowed.

3. **Implement the class `Transcript`.**

   Implement a class `Transcript` that manages a collection of `CourseResult` objects.

   The class must:

   - Own a list of course results (composition),
   - Initialize with an empty list,
   - Prevent external modification of its internal list.

4. **Add course results to the transcript.**

   Implement the method:

   ```
   void addCourseResult(CourseResult r)
   ```

   This method adds a course result to the transcript. The argument must not be `null`.

5. **Compute GPA on a 4.0 scale.**

   Implement the method:

   ```
   double gpa4()
   ```

   GPA is computed as a weighted average based on course credits. Scores are converted from a 10-point scale to a 4-point scale using the following rules:

$$\begin{aligned} \text{score} \geq 8.5 &\rightarrow 4.0 \\ \text{score} \geq 7.0 &\rightarrow 3.0 \\ \text{score} \geq 5.5 &\rightarrow 2.0 \\ \text{score} \geq 4.0 &\rightarrow 1.0 \\ \text{otherwise} &\rightarrow 0.0 \end{aligned}$$

The GPA is then calculated as a weighted average:

$$\text{GPA} = \frac{\sum(\text{point4}(score_i) \cdot credits_i)}{\sum credits_i}$$

Rounding rule: The final GPA must be rounded to *one decimal place* using standard rounding in Java:

```
Math.round(gpa * 10.0) / 10.0
```

If the transcript contains no courses, the method must return `0.0`.

If the transcript contains no course results, the GPA must be `0.0`.

6. **Count failed courses.**

   Implement the method:

   ```
   int failedCount()
   ```

   A course is considered failed if its score is strictly less than 5.0 (e.g., 4.9).

7. **Protect internal data (encapsulation).**

   Implement the method:

   ```
   List<CourseResult> getResults()
   ```

   The returned list must not allow callers to modify the internal state of the transcript. You may return an unmodifiable view or a defensive copy.

   The item order in the returned list must be the same as the item order in the internal list `results`.

Design Constraints

- GPA must not be stored as a field.
- GPA computation logic must reside only in `Transcript`.
- `CourseResult` must implement `GradedItem`.
- Internal collections must not be exposed in a mutable form.

2 Exercise 2: Student Classification (30 points)

Overview

In this exercise, you will implement a student management subsystem based on the UML class diagram shown in Figure 2. The goal is to practice object-oriented design using:

- Abstract classes and inheritance
- Method overriding and polymorphism
- Composition (HAS-A relationship with `Transcript`)
- Encapsulation and responsibility separation

The system models different types of students and their academic status, as well as tuition discount rules that vary by student type. You must follow the structure and constraints specified in the class diagram. Using ad-hoc conditionals (e.g. `instanceof`) instead of polymorphism is not allowed.

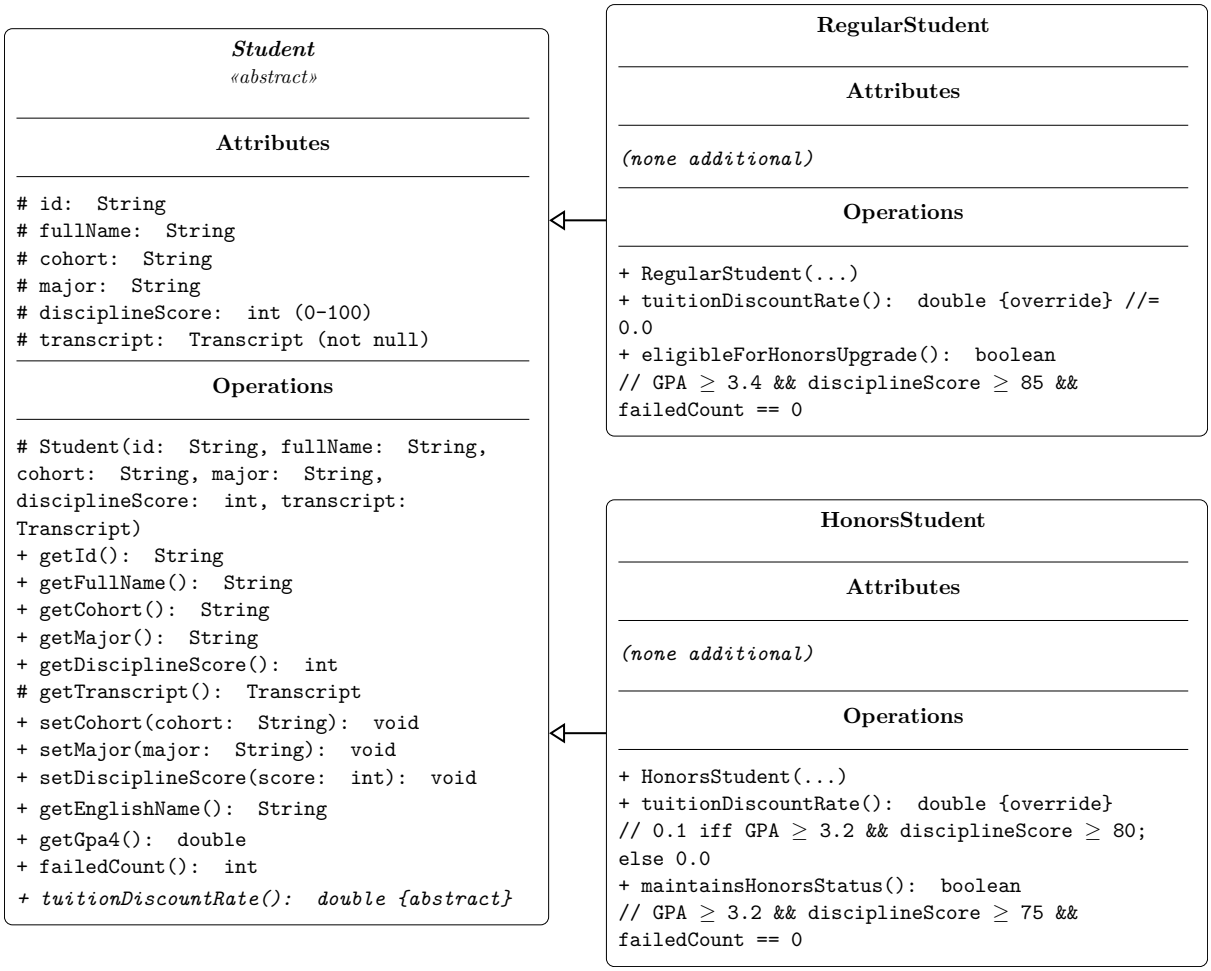

Figure 2: UML class diagram for the Student hierarchy

Tasks

1. **Implement the abstract class `Student`.**

   Create an abstract class `Student` with the following protected attributes:

   - `id : String`
   - `fullName : String`
   - `cohort : String`
   - `major : String`
   - `disciplineScore : int` (range $[0, 100]$)
   - `transcript : Transcript` (must not be `null`)

   All attributes must be properly encapsulated (not public).

2. **Define the constructor and accessors for `Student`.**

   Implement a protected constructor:

   ```
   Student(id, fullName, cohort, major, disciplineScore, transcript)
   ```

   The constructor must initialize all attributes and reject invalid values.

   If any string attribute (e.g., `id`, `fullName`, `cohort`, `major`) contains leading or trailing whitespace, it must be trimmed. For example, `" An Hoa "` → `"An Hoa"`.

   Provide getter methods for all attributes. Provide setter methods only for:

   - `cohort`
   - `major`
   - `disciplineScore`

   The GPA must *not* be stored or modified directly in `Student`.

3. **Implement `getEnglishName()`.**

   Implement the method:

   ```
   String getEnglishName()
   ```

   This method converts a Vietnamese full name into English name order:

   - Family name goes last
   - Given name and middle names come first
   - Middle names are joined using hyphens

   Examples: `"Le Van An"` → `"Van-An Le"`, `"Tran An"` → `"An Tran"`.

   You may assume that each name contains at least two words.

4. **Declare the abstract tuition policy.**

   Declare the abstract method:

```
double tuitionDiscountRate();
```

This method defines the tuition discount rate for a student and must be implemented by all subclasses.

5. **Implement GPA-related helper methods in `Student`.**

   To support classification policies without relying on concrete subclasses, the `Student` class must expose the following GPA-related methods:

   - `double getGpa4()`
   - `int failedCount()`

   Their responsibilities are defined as follows:

   - `getGpa4()` returns the GPA on a 4-point scale by delegating directly to the associated `Transcript` object.
   - `failedCount()` returns the number of failed courses by delegating directly to `Transcript`.

6. **Implement the class `RegularStudent`.**

   Create a class `RegularStudent` that extends `Student`.

   - Override `tuitionDiscountRate()` so that it always returns `0.0`.

7. **Implement `eligibleForHonorsUpgrade()`.**

   Add the method:

```
boolean eligibleForHonorsUpgrade()
```

   A regular student is eligible for an honors upgrade if and only if:

   - GPA ≥ 3.4
   - `disciplineScore` ≥ 85
   - `failedCount() == 0`

   GPA and failed-course information must be obtained via `Transcript`.

8. **Implement the class `HonorsStudent`.**

   Create a class `HonorsStudent` that extends `Student`.

   Override `tuitionDiscountRate()` with the following rule:

   - Return `0.1` if GPA ≥ 3.2 and `disciplineScore` ≥ 80
   - Otherwise, return `0.0`

   GPA must be obtained from `Transcript`. Hard-coding GPA values or storing GPA as a field is not allowed.

9. **Implement `maintainsHonorsStatus()`.**

   Add the method:

   ```
   boolean maintainsHonorsStatus()
   ```

   An honors student maintains honors status if and only if:

   - GPA ≥ 3.2
   - `disciplineScore` ≥ 75
   - `failedCount() == 0`

Design Constraints

- Do not use `instanceof`.
- Do not duplicate GPA computation logic.
- All academic calculations must be delegated to `Transcript`.
- Subclass-specific behavior must be implemented via method overriding.

3 Exercise 3: Student Policies (20 points)

Overview

In this exercise, you will implement a student classification subsystem based on the UML class diagram shown in Figure 3. The goal is to practice object-oriented design using:

- Interfaces and polymorphism
- Strategy-based design
- Inheritance-friendly extension
- Separation of policy and domain logic

The system classifies students into academic categories using interchangeable classification policies. Each policy encapsulates a set of rules, while the classification logic remains independent of concrete student types. You must follow the structure and constraints specified in the class diagram. Using ad-hoc conditionals (e.g. `instanceof`) instead of polymorphism is not allowed.

Tasks

1. **Define the classification result.**

   Define an enumeration `Classification` with the following values:

   - `EXCELLENT`
   - `GOOD`
   - `FAIR`
   - `PASS`
   - `FAIL`

   Each student must be assigned exactly one classification.

2. **Define the `ClassificationPolicy` interface.**

   Create an interface:

   ```
   Classification classify(Student s);
   ```

   The method takes a `Student` object and returns its classification. The implementation must work uniformly for all subclasses of `Student`.

   Policies must not depend on whether a student is a `RegularStudent` or an `HonorsStudent`.

3. **Specify the classification inputs.**

   Classification policies must base their decisions *only* on:

   - `s.getGpa4()`
   - `s.failedCount()`

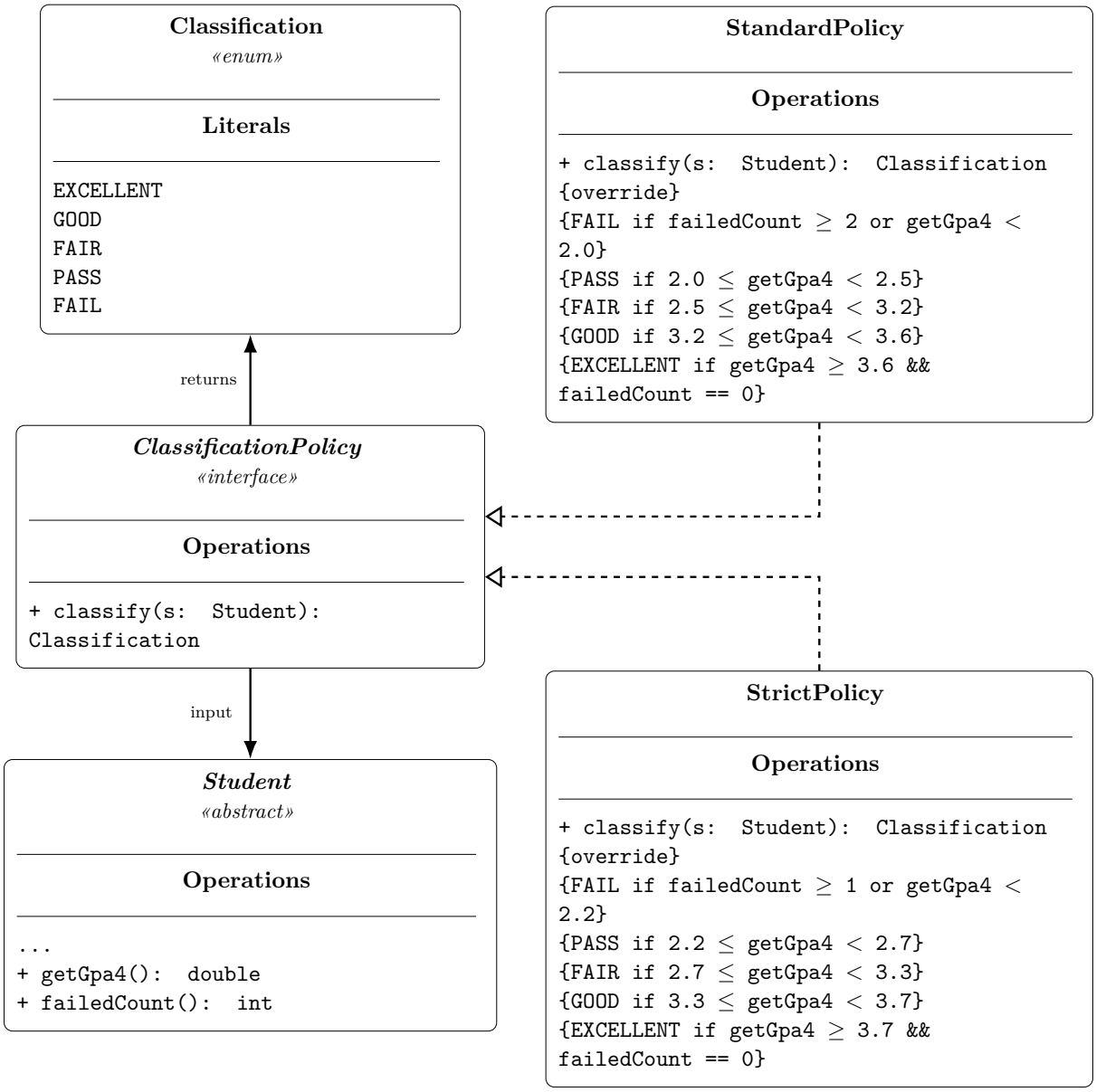

Figure 3: UML class diagram for the classification policy subsystem

`getGpa4()` values must be obtained via `Transcript`. No policy is allowed to modify, recompute, or store `getGpa4()` values.

4. **Implement the `StandardPolicy`.**

   Create a class `StandardPolicy` that implements `ClassificationPolicy`.

   Classification rules are:

   - `FAIL` if `failedCount()` $\geq$ `2` or `getGpa4()` < 2.0
   - `PASS` if $2.0 \leq$ `getGpa4()` < 2.5
   - `FAIR` if $2.5 \leq$ `getGpa4()` < 3.2
   - `GOOD` if $3.2 \leq$ `getGpa4()` < 3.6
   - `EXCELLENT` if `getGpa4()` ≥ 3.6 and `failedCount() == 0`

5. **Implement the `StrictPolicy`.**

   Create a class `StrictPolicy` that also implements `ClassificationPolicy`. This policy applies stricter thresholds:

   - `FAIL` if `failedCount()` $\geq$ `1` or `getGpa4()` < 2.2
   - `PASS` if $2.2 \leq$ `getGpa4()` < 2.7
   - `FAIR` if $2.7 \leq$ `getGpa4()` < 3.3
   - `GOOD` if $3.3 \leq$ `getGpa4()` < 3.7
   - `EXCELLENT` if `getGpa4()` ≥ 3.7 and `failedCount() == 0`

Design Constraints

- Do not use `instanceof` or type casting.
- Policies must rely only on public methods of `Student`.
- Academic calculations must be delegated to `Transcript`.
- Adding a new policy must not require modifying existing code.

4 Exercise 4: Reporting and Sorting (20 points)

Overview

In this exercise, you will implement a reporting and analysis subsystem over a collection of students, based on the UML class diagram shown in Figure 4. The objective is to practice object-oriented design and collection processing, with emphasis on:

- Polymorphic collections (`List<Student>`)
- Stable grouping, sorting, and aggregation
- Clear separation between storage (Repository) and processing (Service)
- Reuse of pluggable classification policies
- Immutable result objects

The system operates on a heterogeneous list of students (regular and honors) exclusively through their common superclass `Student`. All reporting and analysis logic must remain independent of concrete student subtypes.

Tasks

1. **Implement `StudentRepository`.**

   Implement a class `StudentRepository` responsible for storing students.

   - Internally, the repository must store a `List<Student>` that may contain both `RegularStudent` and `HonorsStudent` instances.
   - Implement the method:

     ```
     void add(Student s)
     ```

     to add a student to the repository.
   - Implement the method:

     ```
     List<Student> findAll()
     ```

     which returns either an unmodifiable view or a defensive copy of the internal list.

   The order of students in the returned list must exactly match the insertion order in the repository's internal `students` list. Direct external modification of the internal list must not be possible.

2. **Implement `ReportingService`.**

   Implement a class `ReportingService` that performs analysis and reporting over students obtained from `StudentRepository`.

3. **Group students by classification.**

   Implement the method:

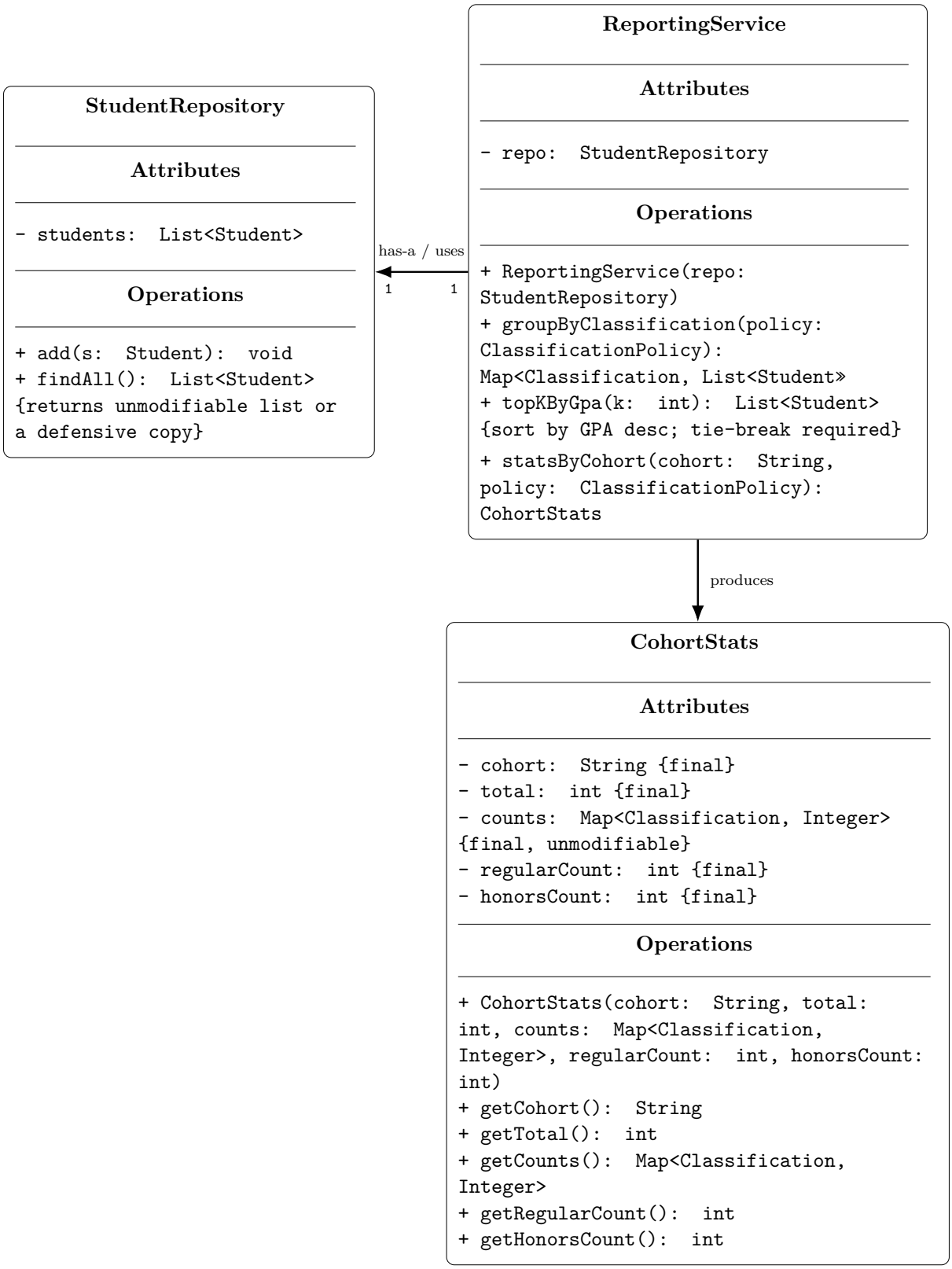

Figure 4: UML class diagram for reporting and sorting over students

```
Map<Classification, List<Student>
groupByClassification(ClassificationPolicy policy)
```

Each student must be classified using the provided `ClassificationPolicy` and grouped according to the classification result.

For each classification group, the resulting list must be *stable* with respect to the repository's internal `students` list. In particular, for any two students A and B belonging to the same classification, if A appears before B in `repo.students`, then A must also appear before B in the corresponding classified list.

4. **Find the top K students by GPA.**

   Implement the method:

```
List<Student> topKByGpa(int k)
```

   Students must be sorted according to the following priority rules:

   (a) Higher GPA (`getGpa4()`) ranks first.
   (b) If GPA values are equal, prefer the student with a higher `disciplineScore`.
   (c) If both GPA and `disciplineScore` are equal, prefer the student with a lower `failedCount`.
   (d) If all criteria above are equal, prefer the student whose English name is lexicographically larger.

   The returned list must contain at most k students.

5. **Implement `CohortStats`.**

   Implement an immutable value class `CohortStats` that represents aggregated statistics for a cohort.

   - The class must store:
     - The cohort identifier
     - The total number of students in the cohort
     - The number of students in each `Classification`
     - The number of `RegularStudents`
     - The number of `HonorsStudents`
   - All fields must be initialized via the constructor.
   - All fields must be immutable after construction.
   - Any collection exposed by this class must be unmodifiable.

6. **Compute statistics for a cohort.**

   Implement the method:

```
CohortStats statsByCohort(String cohort,
        ClassificationPolicy policy)
```

   This method computes statistics for all students belonging to the given cohort, using the provided `ClassificationPolicy`, and returns the result as a fully populated `CohortStats` object.

Design Constraints

Your implementation must satisfy the following constraints:

- The repository must store `List<Student>`, not subtype-specific lists.
- The method `findAll()` must not expose a mutable internal list.
- Reporting logic must not rely on `instanceof` for classification.
- All classification decisions must be delegated to `ClassificationPolicy`.
- `CohortStats` must be immutable and defensively encapsulated.